

Hive & NoSQL for Data Warehousing

Big Data – FI-GLAL-SE5 – Workshop Guide

Prepared by: Fakher Neila

October 4, 2026

1. Objective

Build a mini **Data Warehouse** using Apache Hive on top of a Dockerized Hadoop environment. By the end of this workshop, you will be able to:

- Launch a fully configured Hive environment using Docker.
- Create databases and tables with HiveQL.
- Load CSV data into Hive tables.
- Create **partitioned tables** and understand their physical layout.
- Run analytical queries and observe partition pruning.
- Compare Hive with NoSQL (MongoDB) for Data Warehouse use cases.

Dataset used: Tunisian e-commerce orders (customers, cities, products, prices). used cities: Tunis, Sfax, Sousse, Ariana.

2. Prerequisites

- an OS (CachyOS in our case).
- Terminal access with `sudo` privileges.
- Internet connection (for pulling the Docker image).

3. Part 1 – Install Docker

3.1 Install Docker packages

```
sudo pacman -S docker docker-compose docker-buildx
```

Screenshot:

```

sudo pacman -S docker docker-compose docker-buildx
warning: docker-1:29.8.2-1.1 is up to date -- reinstalling
warning: docker-compose-5.6.0-1.1 is up to date -- reinstalling
warning: docker-buildx-0.37.2-1.1 is up to date -- reinstalling
resolving dependencies...
looking for conflicting packages...

Package (3)                                Old Version  New Version  Net Change
cachyos-extra-v3/docker                    1:29.8.2-1.1 1:29.8.2-1.1 0.00 MiB
cachyos-extra-v3/docker-buildx             0.37.2-1.1   0.37.2-1.1   0.00 MiB
cachyos-extra-v3/docker-compose            5.6.0-1.1    5.6.0-1.1    0.00 MiB

Total Installed Size: 208.57 MiB
Net Upgrade Size:      0.00 MiB

:: Proceed with installation? [Y/n]
(3/3) checking keys in keyring [-----] 100%
(3/3) checking package integrity [-----] 100%
(3/3) loading package files [-----] 100%
(3/3) checking for file conflicts [-----] 100%
:: Running pre-transaction hooks...
(1/2) Performing snapper pre snapshots for the following configurations...
=> root: 740
(2/2) Waiting for limine-snapper-sync to finish...
:: Processing package changes...
(1/3) reinstalling docker [-----] 100%
(2/3) reinstalling docker-compose [-----] 100%
(3/3) reinstalling docker-buildx [-----] 100%
:: Running post-transaction hooks...
(1/6) Creating system user accounts...
(2/6) Reloading system manager configuration...
(3/6) Enqueuing marked services...
(4/6) Arming ConditionNeedsUpdate...
(5/6) Checking which packages need to be rebuilt
(6/6) Performing snapper post snapshots for the following configurations...
=> root: 741

```

3.2 Enable and start the Docker service

```

sudo systemctl enable --now docker
systemctl status docker

```

Screenshot:

```

sudo systemctl enable --now docker
[sudo] password for fakhermetba5er:

systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-10-03 20:32:26 CET; 3h 0min ago
     Invocation: 3c581aaac5bf4016abdbdb183cfa8a27
   TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
    Main PID: 979 (dockerd)
       Tasks: 57
    Memory: 104.1M (peak: 170.7M, swap: 8.1M, swap peak: 21M)
       CPU: 26.307s
    CGroup: /system.slice/docker.service
            └─979 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Oct 03 23:10:35 bennacef-vostro dockerd[979]: time="2026-10-03T23:10:35.135416880+01:00" leve
Oct 03 23:10:35 bennacef-vostro dockerd[979]: time="2026-10-03T23:10:35.135589526+01:00" leve
Oct 03 23:10:35 bennacef-vostro dockerd[979]: time="2026-10-03T23:10:35.135612618+01:00" leve
Oct 03 23:10:42 bennacef-vostro dockerd[979]: time="2026-10-03T23:10:42.211922204+01:00" leve
Oct 03 23:10:42 bennacef-vostro dockerd[979]: time="2026-10-03T23:10:42.237674290+01:00" leve
Oct 03 23:10:51 bennacef-vostro dockerd[979]: time="2026-10-03T23:10:51.865328412+01:00" leve
Oct 03 23:11:23 bennacef-vostro dockerd[979]: time="2026-10-03T23:11:23.934550454+01:00" leve
Oct 03 23:11:33 bennacef-vostro dockerd[979]: time="2026-10-03T23:11:33.777216849+01:00" leve
Oct 03 23:21:31 bennacef-vostro dockerd[979]: time="2026-10-03T23:21:31.515122810+01:00" leve
Oct 03 23:23:52 bennacef-vostro dockerd[979]: time="2026-10-03T23:23:52.727633135+01:00" leve

```

3.3 Add your user to the docker group

```
sudo usermod -aG docker $USER
groups
```

Screenshot:

```

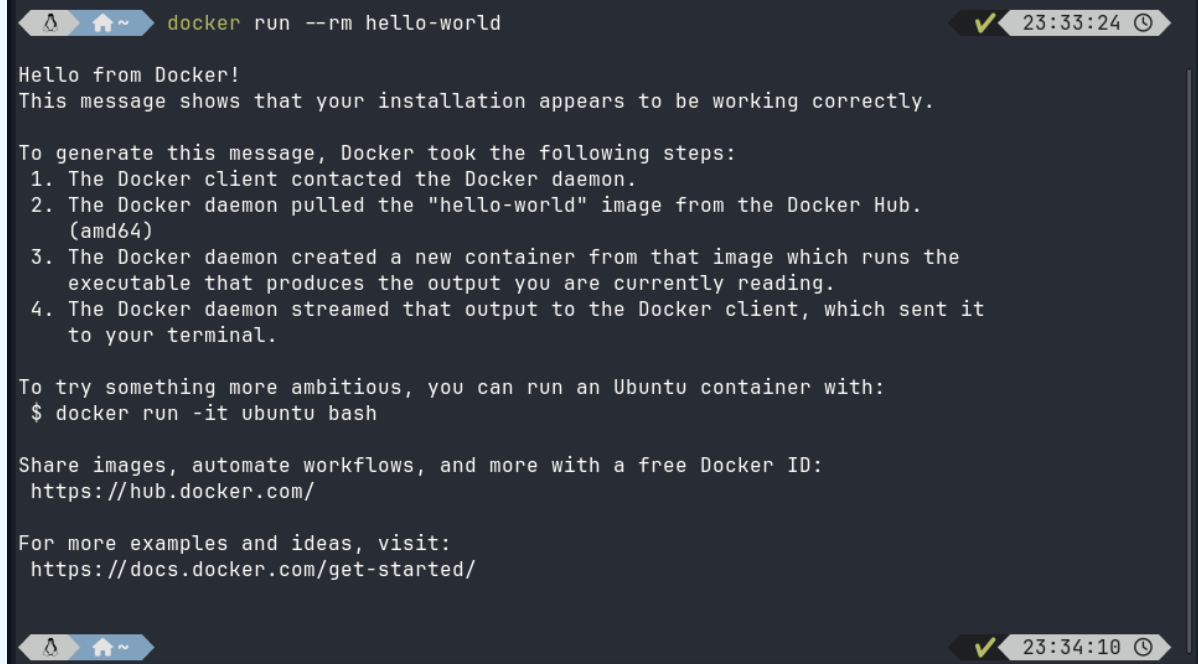
sudo usermod -aG docker $USER

groups
fakhermetba5er sys network adbusers docker nopasswdlogin rfkill users video storage lp audio
heel android-sdk

```

3.4 Test Docker

```
docker run --rm hello-world
```

 Screenshot:

```
docker run --rm hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

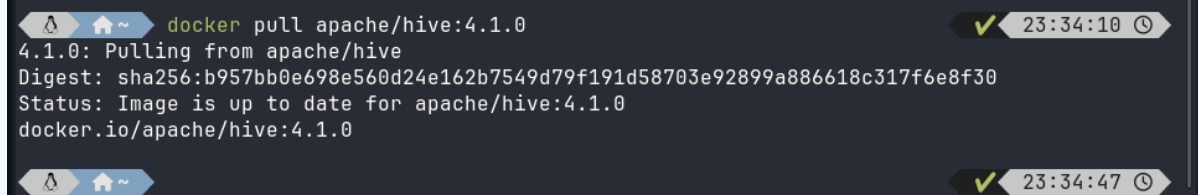
To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

3.5 Pull the Hive image

```
docker pull apache/hive:4.1.0
```

 Screenshot:

```
docker pull apache/hive:4.1.0
4.1.0: Pulling from apache/hive
Digest: sha256:b957bb0e698e560d24e162b7549d79f191d58703e92899a886618c317f6e8f30
Status: Image is up to date for apache/hive:4.1.0
docker.io/apache/hive:4.1.0
```

4. Part 2 – Launch Hive

4.1 Start the Hive container

```
docker run -d \
  -p 10000:10000 \
  -p 10002:10002 \
  --env SERVICE_NAME=hiveserver2 \
  --name hive-workshop \
  apache/hive:4.1.0
```

Screenshot:

```

docker run -d \
  -p 10000:10000 \
  -p 10002:10002 \
  --env SERVICE_NAME=hiveserver2 \
  --name hive-workshop \
  apache/hive:4.1.0
9899de268b47368e7260787deed29cb85285574792d6fbb4739237bdca810a57

```

4.2 Verify the container is running

```
docker ps
```

Screenshot:

```

docker ps

```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	P
9899de268b47	apache/hive:4.1.0	"sh -c /entrypoint.sh"	29 seconds ago	Up 28 seconds	0
.0.0.0:10000→10000/tcp, [::]:10000→10000/tcp, 9083/tcp, 0.0.0.0:10002→10002/tcp, [::]:10002→10002/tcp					
hive-workshop					

4.3 Wait for HiveServer2 to start

```
docker logs -f hive-workshop
```

Wait until you see `Service:HiveServer2` is started. and Web UI has started on port 10002. Then press `Ctrl+C`.

Screenshot:

```

docker logs -f hive-workshop
+ : derby
+ SKIP_SCHEMA_INIT=false
+ [[ ' ' = \t\r\n\u\e ]]
+ export HIVE_CONF_DIR=/opt/hive/conf
+ HIVE_CONF_DIR=/opt/hive/conf
+ '[' -d ' ' ']'
+ export 'HADOOP_CLIENT_OPTS= -Xmx16 '
+ HADOOP_CLIENT_OPTS=' -Xmx16 '
+ [[ false = \f\a\l\s\e ]]
+ initialize_hive
+ COMMAND=-initOrUpgradeSchema
++ echo 4.1.0
++ cut -d . -f1
+ '[' 4 -lt 4 ']'
+ [[ -n ' ' ]]

```

4.4 Connect to Hive with Beeline

```
docker exec -it hive-workshop beeline -u 'jdbc:hive2://localhost:10000/'
```

You should land on:

```
0: jdbc:hive2://localhost:10000/>
```

Screenshot:

```

docker exec -it hive-workshop beeline -u 'jdbc:hive2://localhost:10000/'
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/opt/hive/lib/log4j-slf4j-impl-2.24.3.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/opt/hadoop/share/hadoop/common/lib/slf4j-reload4j-1.7.36.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/opt/hive/lib/log4j-slf4j-impl-2.24.3.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/opt/hadoop/share/hadoop/common/lib/slf4j-reload4j-1.7.36.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
2026-10-03T22:37:39.510222593Z main WARN The use of package scanning to locate Log4j plugins is deprecated.
Please remove the `packages` attribute from your configuration file.
See https://logging.apache.org/log4j/2.x/faq.html#package-scanning for details.
2026-10-03T22:37:39.580397725Z main INFO Starting configuration org.apache.logging.log4j.core.config.properties.PropertiesConfiguration@436c81a3...
2026-10-03T22:37:39.580583416Z main INFO Start watching for changes to jar:file:/opt/hive/lib/hive-beeline-4.1.0.jar!/beeline-log4j2.properties every 0 seconds
2026-10-03T22:37:39.580759781Z main INFO Configuration org.apache.logging.log4j.core.config.properties.PropertiesConfiguration@436c81a3 started.
2026-10-03T22:37:39.582483059Z main INFO Stopping configuration org.apache.logging.log4j.core.config.DefaultConfiguration@2df9b86...
2026-10-03T22:37:39.582898764Z main INFO Configuration org.apache.logging.log4j.core.config.DefaultConfiguration@2df9b86 stopped.
Connecting to jdbc:hive2://localhost:10000/
Connected to: Apache Hive (version 4.1.0)
Driver: Hive JDBC (version 4.1.0)
Transaction isolation: TRANSACTION_REPEATABLE_READ
Beeline version 4.1.0 by Apache Hive
0: jdbc:hive2://localhost:10000/>

```

5. Part 3 – Prepare the Dataset

5.1 Create the CSV file on the host

```
nano ~/orders.csv
```

Paste the following content (no header row):

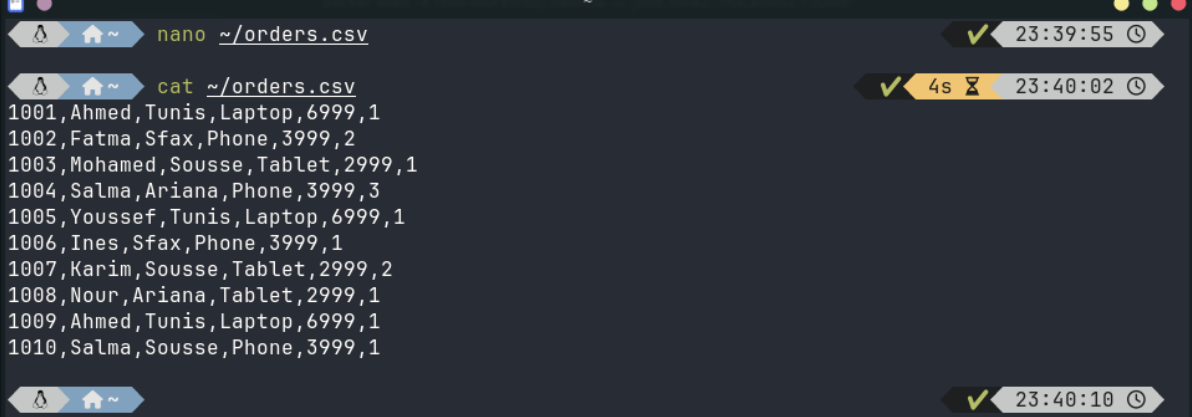
```

1001,Ahmed,Tunis,Laptop,6999,1
1002,Fatma,Sfax,Phone,3999,2
1003,Mohamed,Sousse,Tablet,2999,1
1004,Salma,Ariana,Phone,3999,3
1005,Youssef,Tunis,Laptop,6999,1
1006,Ines,Sfax,Phone,3999,1
1007,Karim,Sousse,Tablet,2999,2
1008,Nour,Ariana,Tablet,2999,1
1009,Ahmed,Tunis,Laptop,6999,1
1010,Salma,Sousse,Phone,3999,1

```

Save with Ctrl+O, Enter, then Ctrl+X.

Screenshot:



```

nano ~/orders.csv
✓ 23:39:55

cat ~/orders.csv
✓ 4s 23:40:02
1001,Ahmed,Tunis,Laptop,6999,1
1002,Fatma,Sfax,Phone,3999,2
1003,Mohamed,Sousse,Tablet,2999,1
1004,Salma,Ariana,Phone,3999,3
1005,Youssef,Tunis,Laptop,6999,1
1006,Ines,Sfax,Phone,3999,1
1007,Karim,Sousse,Tablet,2999,2
1008,Nour,Ariana,Tablet,2999,1
1009,Ahmed,Tunis,Laptop,6999,1
1010,Salma,Sousse,Phone,3999,1
✓ 23:40:10

```

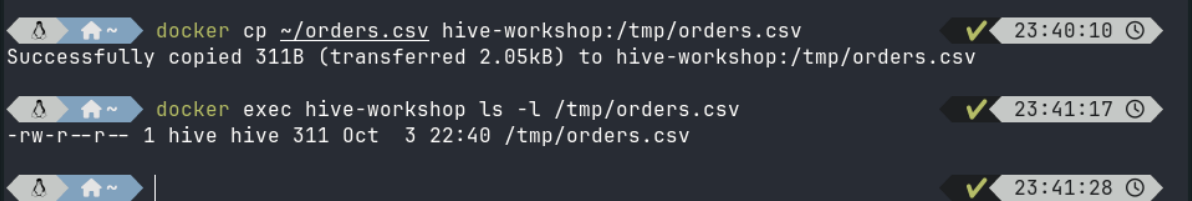
5.2 Copy the CSV into the container

```

docker cp ~/orders.csv hive-workshop:/tmp/orders.csv
docker exec hive-workshop ls -l /tmp/orders.csv

```

Screenshot:



```

docker cp ~/orders.csv hive-workshop:/tmp/orders.csv
✓ 23:40:10
Successfully copied 311B (transferred 2.05kB) to hive-workshop:/tmp/orders.csv

docker exec hive-workshop ls -l /tmp/orders.csv
✓ 23:41:17
-rw-r--r-- 1 hive hive 311 Oct 3 22:40 /tmp/orders.csv
✓ 23:41:28

```

6. Part 4 – Create Database and Table

All SQL commands from this point run inside Beeline.

6.1 Create the database

```

CREATE DATABASE IF NOT EXISTS ecommerce_dw;
USE ecommerce_dw;

```

 Screenshot:

```

Connecting to jdbc:hive2://localhost:10000/
Connected to: Apache Hive (version 4.1.0)
Driver: Hive JDBC (version 4.1.0)
Transaction isolation: TRANSACTION_REPEATABLE_READ
Beeline version 4.1.0 by Apache Hive
0: jdbc:hive2://localhost:10000/> CREATE DATABASE IF NOT EXISTS ecommerce_dw;
INFO : Compiling command(queryId=hive_20261003224225_ca92f06e-e98b-47da-a591-c59c7b0033dd): C
REATE DATABASE IF NOT EXISTS ecommerce_dw
INFO : Semantic Analysis Completed (retrial = false)
INFO : Created Hive schema: Schema(fieldSchemas:null, properties:null)
INFO : Completed compiling command(queryId=hive_20261003224225_ca92f06e-e98b-47da-a591-c59c7b
0033dd); Time taken: 0.673 seconds
INFO : Concurrency mode is disabled, not creating a lock manager
INFO : Executing command(queryId=hive_20261003224225_ca92f06e-e98b-47da-a591-c59c7b0033dd): C
REATE DATABASE IF NOT EXISTS ecommerce_dw
INFO : Starting task [Stage-0:DDL] in serial mode
INFO : Completed executing command(queryId=hive_20261003224225_ca92f06e-e98b-47da-a591-c59c7b
0033dd); Time taken: 0.038 seconds
No rows affected (0.922 seconds)
0: jdbc:hive2://localhost:10000/> USE ecommerce_dw;
INFO : Compiling command(queryId=hive_20261003224244_52342438-361a-4348-8b39-8f41be2dc51b): U
SE ecommerce_dw
INFO : Semantic Analysis Completed (retrial = false)
INFO : Created Hive schema: Schema(fieldSchemas:null, properties:null)
INFO : Completed compiling command(queryId=hive_20261003224244_52342438-361a-4348-8b39-8f41be
2dc51b); Time taken: 0.018 seconds
INFO : Concurrency mode is disabled, not creating a lock manager
INFO : Executing command(queryId=hive_20261003224244_52342438-361a-4348-8b39-8f41be2dc51b): U
SE ecommerce_dw
INFO : Starting task [Stage-0:DDL] in serial mode
INFO : Completed executing command(queryId=hive_20261003224244_52342438-361a-4348-8b39-8f41be
2dc51b); Time taken: 0.006 seconds
No rows affected (0.033 seconds)
0: jdbc:hive2://localhost:10000/>

```

6.2 Create the orders table

```

CREATE TABLE orders (
  order_id INT,
  customer STRING,
  city STRING,
  product STRING,
  price FLOAT,
  quantity INT
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE;

```

 Screenshot:

```

0: jdbc:hive2://localhost:10000/> CREATE TABLE orders (
. . . . . > order_id INT,
. . . . . > customer STRING,
. . . . . > city STRING,
. . . . . > product STRING,
. . . . . > price FLOAT,
. . . . . > quantity INT
. . . . . > )
. . . . . > ROW FORMAT DELIMITED
. . . . . > FIELDS TERMINATED BY ','
. . . . . > STORED AS TEXTFILE;
Error: Error while compiling statement: FAILED: ParseException line 10:22 mismatched input ',',
expecting StringLiteral near 'BY' in table row format's field separator; Query ID: hive_20261
003224459_e4e94f97-0c54-4e8f-903d-0a53d24ffe05 (state=42000,code=40000)
0: jdbc:hive2://localhost:10000/> STORED AS TEXTFILE;
Error: Error while compiling statement: FAILED: ParseException line 1:0 cannot recognize input
near 'STORED' 'AS' 'TEXTFILE'; Query ID: hive_20261003224509_ba9fe8f9-ef51-40a6-b1d0-be0662b0
7c49 (state=42000,code=40000)
0: jdbc:hive2://localhost:10000/> CREATE TABLE orders (
. . . . . > order_id INT,
. . . . . > customer STRING,
. . . . . > city STRING,
. . . . . > product STRING,
. . . . . > price FLOAT,
. . . . . > quantity INT
. . . . . > )
. . . . . > ROW FORMAT DELIMITED
. . . . . > FIELDS TERMINATED BY ','
. . . . . > STORED AS TEXTFILE;

```

6.3 Verify the table

```
SHOW TABLES;
```

 Screenshot:

```

0: jdbc:hive2://localhost:10000/> SHOW TABLES;
INFO : Compiling command(queryId=hive_20261003225008_a401a9e0-798b-436a-8875-ba759af54bd7): S
HOW TABLES
INFO : Semantic Analysis Completed (retrial = false)
INFO : Created Hive schema: Schema(fieldSchemas:[FieldSchema(name:tab_name, type:string, comm
ent:from deserializer)], properties:null)
INFO : Completed compiling command(queryId=hive_20261003225008_a401a9e0-798b-436a-8875-ba759a
f54bd7); Time taken: 0.005 seconds
INFO : Concurrency mode is disabled, not creating a lock manager
INFO : Executing command(queryId=hive_20261003225008_a401a9e0-798b-436a-8875-ba759af54bd7): S
HOW TABLES
INFO : Starting task [Stage-0:DDL] in serial mode
INFO : Completed executing command(queryId=hive_20261003225008_a401a9e0-798b-436a-8875-ba759a
f54bd7); Time taken: 0.021 seconds
+-----+
| tab_name |
+-----+
| orders   |
+-----+
1 row selected (0.046 seconds)
0: jdbc:hive2://localhost:10000/>

```

7. Part 5 – Load the Data

7.1 Load the CSV into Hive

```
LOAD DATA LOCAL INPATH '/tmp/orders.csv' INTO TABLE orders;
```

 Screenshot:

```
0: jdbc:hive2://localhost:10000/> LOAD DATA LOCAL INPATH '/tmp/orders.csv' INTO TABLE orders;
INFO : Compiling command(queryId=hive_20261003225330_7e973391-b993-4bfa-83e1-be55915669b0): L
OAD DATA LOCAL INPATH '/tmp/orders.csv' INTO TABLE orders
INFO : Semantic Analysis Completed (retrial = false)
INFO : Created Hive schema: Schema(fieldSchemas:null, properties:null)
INFO : Completed compiling command(queryId=hive_20261003225330_7e973391-b993-4bfa-83e1-be5591
5669b0); Time taken: 0.126 seconds
INFO : Concurrency mode is disabled, not creating a lock manager
INFO : Executing command(queryId=hive_20261003225330_7e973391-b993-4bfa-83e1-be55915669b0): L
OAD DATA LOCAL INPATH '/tmp/orders.csv' INTO TABLE orders
INFO : Starting task [Stage-0:MOVE] in serial mode
INFO : Loading data to table ecommerce_dw.orders from file:/tmp/orders.csv
INFO : Time taken to move files: 0 ms
INFO : Starting task [Stage-1:STATS] in serial mode
INFO : Executing stats task
INFO : Table ecommerce_dw.orders stats: [numFiles=1, numRows=0, totalSize=311, rawDataSize=0,
numFilesErasureCoded=0]
INFO : StatsTask took 50
INFO : Completed executing command(queryId=hive_20261003225330_7e973391-b993-4bfa-83e1-be5591
5669b0); Time taken: 0.17 seconds
No rows affected (0.305 seconds)
0: jdbc:hive2://localhost:10000/>
```

7.2 Display all rows

```
SELECT * FROM orders;
```

 Screenshot:

```
+-----+-----+-----+-----+-----+-----+
| orders.order_id | orders.customer | orders.city | orders.product | orders.price | order
s.quantity |
+-----+-----+-----+-----+-----+-----+
| 1001           | Ahmed          | Tunis      | Laptop         | 6999.0       | 1
| 1002           | Fatma          | Sfax       | Phone          | 3999.0       | 2
| 1003           | Mohamed        | Sousse     | Tablet         | 2999.0       | 1
| 1004           | Salma          | Ariana     | Phone          | 3999.0       | 3
| 1005           | Youssef        | Tunis      | Laptop         | 6999.0       | 1
| 1006           | Ines           | Sfax       | Phone          | 3999.0       | 1
| 1007           | Karim          | Sousse     | Tablet         | 2999.0       | 2
| 1008           | Nour           | Ariana     | Tablet         | 2999.0       | 1
| 1009           | Ahmed          | Tunis      | Laptop         | 6999.0       | 1
| 1010           | Salma          | Sousse     | Phone          | 3999.0       | 1
+-----+-----+-----+-----+-----+-----+
10 rows selected (1.126 seconds)
```

7.3 Verify the row count

```
SELECT COUNT(*) FROM orders;
```

 Screenshot:

```
+-----+
| _c0 |
+-----+
| 10 |
+-----+
1 row selected (2.716 seconds)
```

8. Part 6 – Create the Partitioned Table

8.1 Enable dynamic partitioning

```
SET hive.exec.dynamic.partition = true;
SET hive.exec.dynamic.partition.mode = nonstrict;
```

 Screenshot:

```
0: jdbc:hive2://localhost:10000/> SET hive.exec.dynamic.partition = true;
No rows affected (0.014 seconds)
0: jdbc:hive2://localhost:10000/> SET hive.exec.dynamic.partition.mode = nonstrict;
No rows affected (0.005 seconds)
0: jdbc:hive2://localhost:10000/> █
```

8.2 Create the partitioned table

```
CREATE TABLE orders_partitioned (
  order_id INT,
  customer STRING,
  product STRING,
  price FLOAT,
  quantity INT
)
PARTITIONED BY (city STRING)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ',';
```

 Screenshot:

```

0: jdbc:hive2://localhost:10000/> CREATE TABLE orders_partitioned (
. . . . . > order_id INT,
. . . . . > customer STRING,
. . . . . > product STRING,
. . . . . > price FLOAT,
. . . . . > quantity INT
. . . . . > )
. . . . . > PARTITIONED BY (city STRING)
. . . . . > ROW FORMAT DELIMITED FIELDS TERMINATED BY ',';
INFO : Compiling command(queryId=hive_20261003225850_0cac6960-95d8-4ef6-aedf-9220b51eb945): C
REATE TABLE orders_partitioned (
order_id INT,
customer STRING,
product STRING,
price FLOAT,
quantity INT
)
PARTITIONED BY (city STRING)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
INFO : Semantic Analysis Completed (retrial = false)
INFO : Created Hive schema: Schema(fieldSchemas:null, properties:null)
INFO : Completed compiling command(queryId=hive_20261003225850_0cac6960-95d8-4ef6-aedf-9220b5
1eb945); Time taken: 0.021 seconds
INFO : Concurrency mode is disabled, not creating a lock manager
INFO : Executing command(queryId=hive_20261003225850_0cac6960-95d8-4ef6-aedf-9220b51eb945): C
REATE TABLE orders_partitioned (
order_id INT,
customer STRING,
product STRING,
price FLOAT,
quantity INT
)
PARTITIONED BY (city STRING)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
INFO : Starting task [Stage-0:DDL] in serial mode
INFO : Completed executing command(queryId=hive_20261003225850_0cac6960-95d8-4ef6-aedf-9220b5
1eb945); Time taken: 0.028 seconds
No rows affected (0.058 seconds)
0: jdbc:hive2://localhost:10000/> 

```

8.3 Insert data with dynamic partitioning

```

INSERT OVERWRITE TABLE orders_partitioned PARTITION(city)
SELECT order_id, customer, product, price, quantity, city
FROM orders;

```

==> 4 dynamic partitions created

 Screenshot:

```

INFO : Partition {city=Ariana} stats: [numFiles=1, numRows=2, totalSize=52, rawDataSize=50, n
umFilesErasureCoded=0]
INFO : Partition {city=Sfax} stats: [numFiles=1, numRows=2, totalSize=51, rawDataSize=49, num
FilesErasureCoded=0]
INFO : Partition {city=Sousse} stats: [numFiles=1, numRows=3, totalSize=82, rawDataSize=79, n
umFilesErasureCoded=0]
INFO : Partition {city=Tunis} stats: [numFiles=1, numRows=3, totalSize=83, rawDataSize=80, nu

```

9. Part 7 – Analytical Queries

9.1 Revenue per city

```

SELECT city, ROUND(SUM(price * quantity), 2) AS revenue
FROM orders_partitioned

```

```
GROUP BY city
ORDER BY revenue DESC;
```

Screenshot:

```
INFO : RAW_INPUT_SPLITS_Map_1: 4
INFO : Completed executing command(queryId=hive_20261003230058_b7b6c52b-54b0-4266-be91-856bf2
ae51e5); Time taken: 0.851 seconds
+-----+-----+
| city | revenue |
+-----+-----+
| Tunis | 20997.0 |
| Ariana | 14996.0 |
| Sousse | 12996.0 |
| Sfax | 11997.0 |
+-----+-----+
4 rows selected (1.21 seconds)
0: jdbc:hive2://localhost:10000/> █
```

9.2 Customers with more than one order

```
SELECT customer, COUNT(*) AS order_count
FROM orders
GROUP BY customer
HAVING COUNT(*) > 1;
```

Screenshot:

```
INFO : INPUT_DIRECTORIES_Map_1: 1
INFO : INPUT_FILES_Map_1: 1
INFO : RAW_INPUT_SPLITS_Map_1: 1
INFO : Completed executing command(queryId=hive_20261003230145_9d7231ab-c2bf-45b7-9676-35e970
d84884); Time taken: 0.579 seconds
+-----+-----+
| customer | order_count |
+-----+-----+
| Ahmed | 2 |
| Salma | 2 |
+-----+-----+
2 rows selected (0.791 seconds)
0: jdbc:hive2://localhost:10000/> █
```

9.3 Partition pruning demo

```
SELECT * FROM orders_partitioned WHERE city = 'Tunis';
```

3 rows (order IDs 1001, 1005, 1009).

Screenshot:

```
+-----+-----+-----+-----+-----+-----+
| orders_partitioned.order_id | orders_partitioned.customer | orders_partitioned.product | orders_partitioned.price | orders_partitioned.quantity | orders_partitioned.city |
+-----+-----+-----+-----+-----+-----+
| 1001 | Ahmed | Laptop | 6999.0 | 1 | Tunis |
| 1005 | Youssef | Laptop | 6999.0 | 1 | Tunis |
| 1009 | Ahmed | Laptop | 6999.0 | 1 | Tunis |
+-----+-----+-----+-----+-----+-----+
3 rows selected (0.296 seconds)
```

10. Part 8 – Verify the Physical Partition Layout

Exit Beeline first:

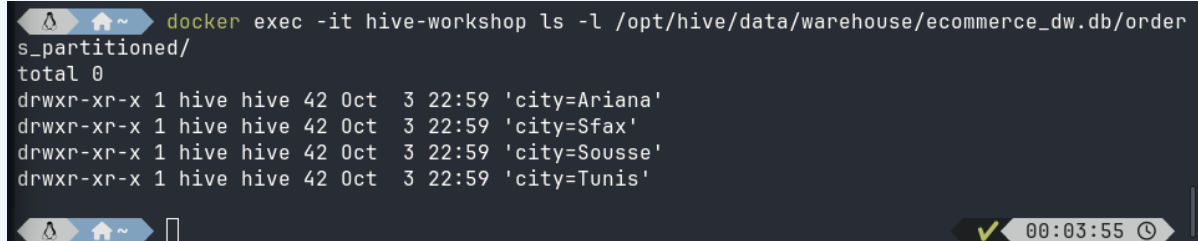
```
!quit
```

Then, on the **host terminal**:

10.1 List the partition directories

```
docker exec -it hive-workshop ls -l /opt/hive/data/warehouse/ecommerce_dw.db/
orders_partitioned/
```

Screenshot:



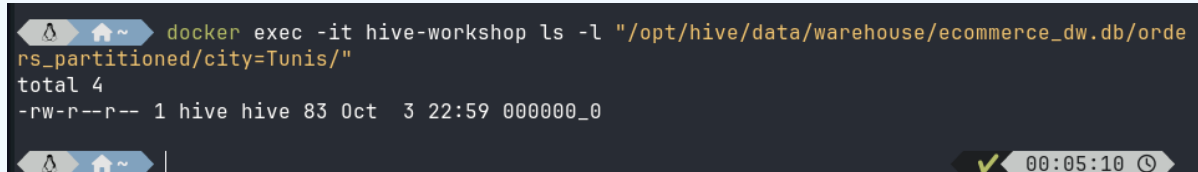
```
docker exec -it hive-workshop ls -l /opt/hive/data/warehouse/ecommerce_dw.db/order
s_partitioned/
total 0
drwxr-xr-x 1 hive hive 42 Oct  3 22:59 'city=Ariana'
drwxr-xr-x 1 hive hive 42 Oct  3 22:59 'city=Sfax'
drwxr-xr-x 1 hive hive 42 Oct  3 22:59 'city=Sousse'
drwxr-xr-x 1 hive hive 42 Oct  3 22:59 'city=Tunis'
```

10.2 Inspect the Tunis partition

```
docker exec -it hive-workshop ls -l "/opt/hive/data/warehouse/ecommerce_dw.db/
orders_partitioned/city=Tunis/"
```

a single file 000000_0.

Screenshot:

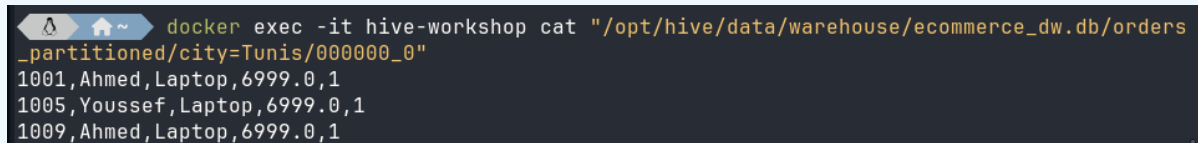


```
docker exec -it hive-workshop ls -l "/opt/hive/data/warehouse/ecommerce_dw.db/orde
rs_partitioned/city=Tunis/"
total 4
-r--r--r-- 1 hive hive 83 Oct  3 22:59 000000_0
```

10.3 Read the raw data inside the Tunis partition

```
docker exec -it hive-workshop cat "/opt/hive/data/warehouse/ecommerce_dw.db/
orders_partitioned/city=Tunis/000000_0"
```

Screenshot:

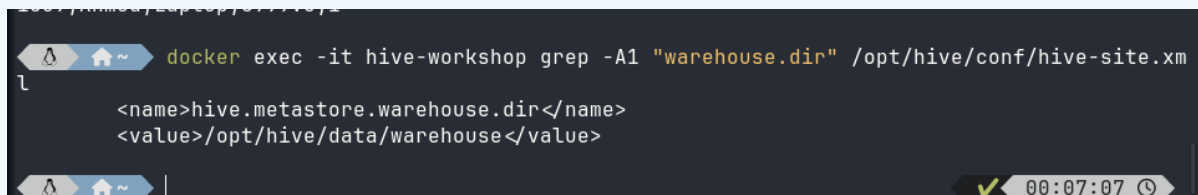


```
docker exec -it hive-workshop cat "/opt/hive/data/warehouse/ecommerce_dw.db/orders
_partitioned/city=Tunis/000000_0"
1001,Ahmed,Laptop,6999.0,1
1005,Youssef,Laptop,6999.0,1
1009,Ahmed,Laptop,6999.0,1
```

10.4 Confirm the warehouse directory config

```
docker exec -it hive-workshop grep -A1 "warehouse.dir" /opt/hive/conf/hive-site.xml
```

Screenshot:



```
docker exec -it hive-workshop grep -A1 "warehouse.dir" /opt/hive/conf/hive-site.xml
<name>hive.metastore.warehouse.dir</name>
<value>/opt/hive/data/warehouse</value>
```

11. Part 9 – Cleanup

At the end of the session:

```
docker stop hive-workshop
docker rm hive-workshop
```

Persistence option: to keep the data across sessions, restart with a named volume:

```
docker run -d -p 10000:10000 -p 10002:10002 \
-v hive-warehouse:/opt/hive/data \
--env SERVICE_NAME=hiveserver2 \
--name hive-workshop apache/hive:4.1.0
```

12. Key Takeaways

- A **Hive table** is a directory on the storage backend.
- A **partition** is a subdirectory named `column=value`.
- Partition pruning lets Hive skip entire directories during a query.
- Hive is optimized for **batch analytics**, not real-time lookups.
- **NoSQL** (MongoDB) complements Hive for schema-free ingestion and fast point queries.